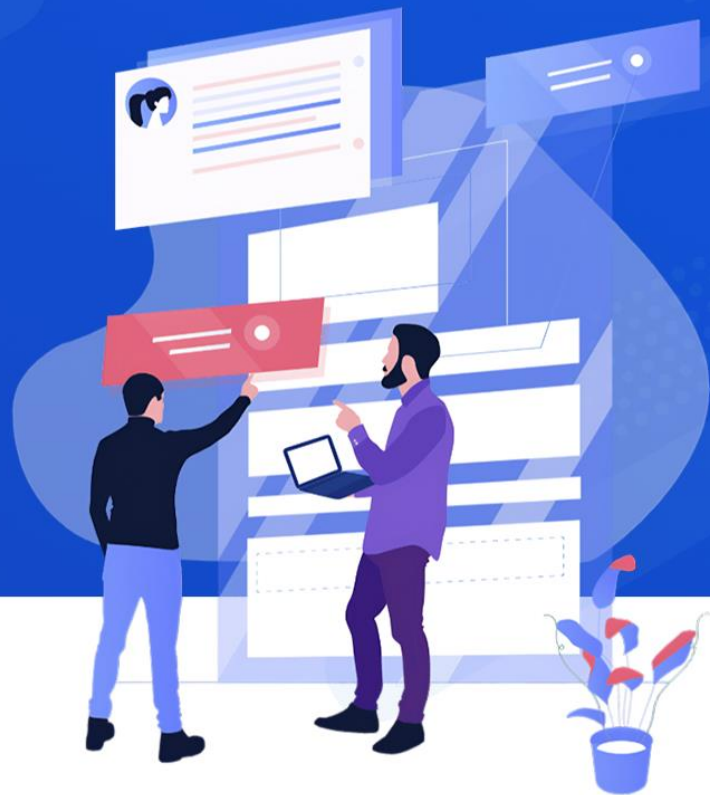


银行客户数据 预测模型选择 与实现

逻辑回归与LightGBM模型

汇报人: 潘奕裴



目录

CONTENT



01 题目

02 模型选择

03 数据预处理

04 训练模型及优化

05 复杂特征关系问题

06 数据分布失衡问题

07 模型评估

01

题目



题目

以银行产品认购预测为背景，预测客户是否会购买银行的产品。

字段	说明
age	年龄
job	职业: admin, unknown, unemployed, management...
marital	婚姻: married, divorced, single
default	信用卡是否有违约: yes or no
housing	是否有房贷: yes or no
contact	联系方式: unknown, telephone, cellular
month	上一次联系的月份: jan, feb, mar, ...
day_of_week	上一次联系的星期几: mon, tue, wed, thu, fri
duration	上一次联系的时长 (秒)
campaign	活动期间联系客户的次数
pdays	上一次与客户联系后的间隔天数

previous	在本次营销活动前，与客户联系的次数
poutcome	之前营销活动的结果: unknown, other, failure, success
emp_var_rate	就业变动率 (季度指标)
cons_price_index	消费者价格指数 (月度指标)
cons_conf_index	消费者信心指数 (月度指标)
lending_rate3m	银行同业拆借率 3个月利率 (每日指标)
nr_employed	雇员人数 (季度指标)
subscribe	客户是否进行购买: yes 或 no

评价标准: AUC值



02

模型选择



Logistic回归——对特征的线性组合

01

02

03

Logistic回归模型

对输入**特征线性组合**，然后使用 $\sigma(z) = \frac{1}{1+e^{-z}}$ 将其转换为概率值，该概率值表示目标变量为正类的概率，是处理二分类问题的核心机制。

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

损失函数与优化算法

使用**交叉熵损失函数**来衡量预测概率分布与实际标签之间的差异，并通过**SAGA随机下降梯度算法**调整模型参数，以最小化这一损失。

正则化

为了防止过拟合，加入正则化项约束模型参数，从而提高模型的泛化能力。**L2正则化**是通过加入参数的平方进行惩罚。

$$L_{\text{final}} = L(\theta) + \lambda \sum_{j=1}^p \theta_j^2$$

基于梯度提升树（**GBDT**）的高效实现，通过迭代构建多棵**决策树来逐步优化**模型的预测结果，但容易过拟合。

处理复杂非线性关系

LightGBM在处理复杂**非线性关系**的数据时表现出色，可以直接将类别特征作为输入，避免了独热编码带来的特征维度爆炸问题，减少了内存占用和计算复杂度。

带深度限制的叶子生长策略

采用的是**带深度限制的叶子生长策略**，优先选择当前对损失函数降低贡献最大的叶子节点进行分裂生长，从而在相对较少的树的数量和较低的计算成本下，达到较好的预测性能。



03

数据预处理



数据预处理

原始数据行数: 22500

去除重复行后的数据行数: 22500

处理缺失值

通过`df.isnull().sum()`方法快速识别出数据集中每个列的缺失值数量。

每列缺失值的数量:

```
id          0
age         0
job         0
marital     0
education   0
default     0
housing     0
loan        0
contact     0
month       0
day_of_week 0
```

```
duration    0
campaign    0
pdays      0
previous    0
poutcome    0
emp_var_rate 0
cons_price_index 0
cons_conf_index 0
lending_rate3m 0
nr_employed 0
subscribe   0
dtype: int64
```

处理重复行

使用`df.drop_duplicates()`方法去除这些重复行。

处理离群点

对于数值类数据设置阈值处理离群点, 如10倍标准差

```
for col in Nu_feature:
    # 计算该列的 10 倍标准差作为阈值
    mean = df[col].mean()
    std = df[col].std()
    threshold = mean + 10 * std

    # 对超出阈值的值进行裁剪
    df[col] = df[col].clip(upper=threshold)
```

数据预处理

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[Nu_feature] = scaler.fit_transform(df[Nu_feature])
```

查看数据信息

使用df.info() 检查数据集的基本信息（如数据类型等）有助于判断是否需要进一步的数据清洗或转换，发现有object数据类型。

```
16 emp_var_rate      22500 non-null float64
17 cons_price_index  22500 non-null float64
18 cons_conf_index   22500 non-null float64
19 lending_rate3m     22500 non-null float64
20 nr_employed        22500 non-null float64
21 subscribe          22500 non-null object
dtypes: float64(5), int64(6), object(11)
```

数据归一化

使用MinMaxScaler函数对数值类数据进行归一化。

处理分类变量

使用LabelEncoder

```
[22500 rows x 11 columns]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
['divorced' 'married' 'single' 'unknown']
['basic.4y' 'basic.6y' 'basic.9y' 'high.school' 'illiterate'
| 'professional.course' 'university.degree' 'unknown']
['no' 'unknown' 'yes']
['no' 'unknown' 'yes']
['no' 'unknown' 'yes']
['cellular' 'telephone']
['apr' 'aug' 'dec' 'jul' 'jun' 'mar' 'may' 'nov' 'oct' 'sep']
['fri' 'mon' 'thu' 'tue' 'wed']
['failure' 'nonexistent' 'success']
```

数据预处理

查看分类数据分布情况

通过value_counts()方法的结果，可以直观地了解“subscribe”特征在不同取值上的分布情况。可以看出来结果失衡。

```
subscribe
no      19548
yes      2952
Name: count, dtype: int64
```

数据分割

将数据集分为训练集和测试集（70%训练集30%），能够评估模型的泛化能力，确保模型在未知数据上的表现。

```
train, test = train_test_split(df, test_size=0.3, random_state=42)

# 特征和目标变量
X_resampled = train.drop(columns=['id', 'subscribe'])
Y_resampled = train['subscribe']
```

04

训练模型及优化



Logistic回归基础拟合

模型选择与训练策略

我们采纳了结合saga优化器的随机平均梯度下降算法，并辅以L2正则化措施，以此平衡模型复杂度与泛化能力。

```
1 #训练模型，saga随机平均下降梯度，l2岭回归
2 estimator = LogisticRegression()
3 solver='saga',penalty='l2',c=1.0,max_iter=10000
4 #开始训练
5 estimator.fit(x_train,y_train)
```

模型评估与结果分析

模型训练得到的AUC值仅为0.77，表明模型具有一定的区分正负样本的能力，但仍需进一步优化以提高准确率和召回率。

	precision	recall	f1-score	support
0	0.87	0.99	0.93	3899
1	0.51	0.04	0.07	601
accuracy			0.87	4500
macro avg	0.69	0.52	0.50	4500
weighted avg	0.82	0.87	0.81	4500

很明显的看到正类召回率为0.04，表明模型在识别正类样本方面表现欠佳。主要原因是数据集中的正类样本比例较低

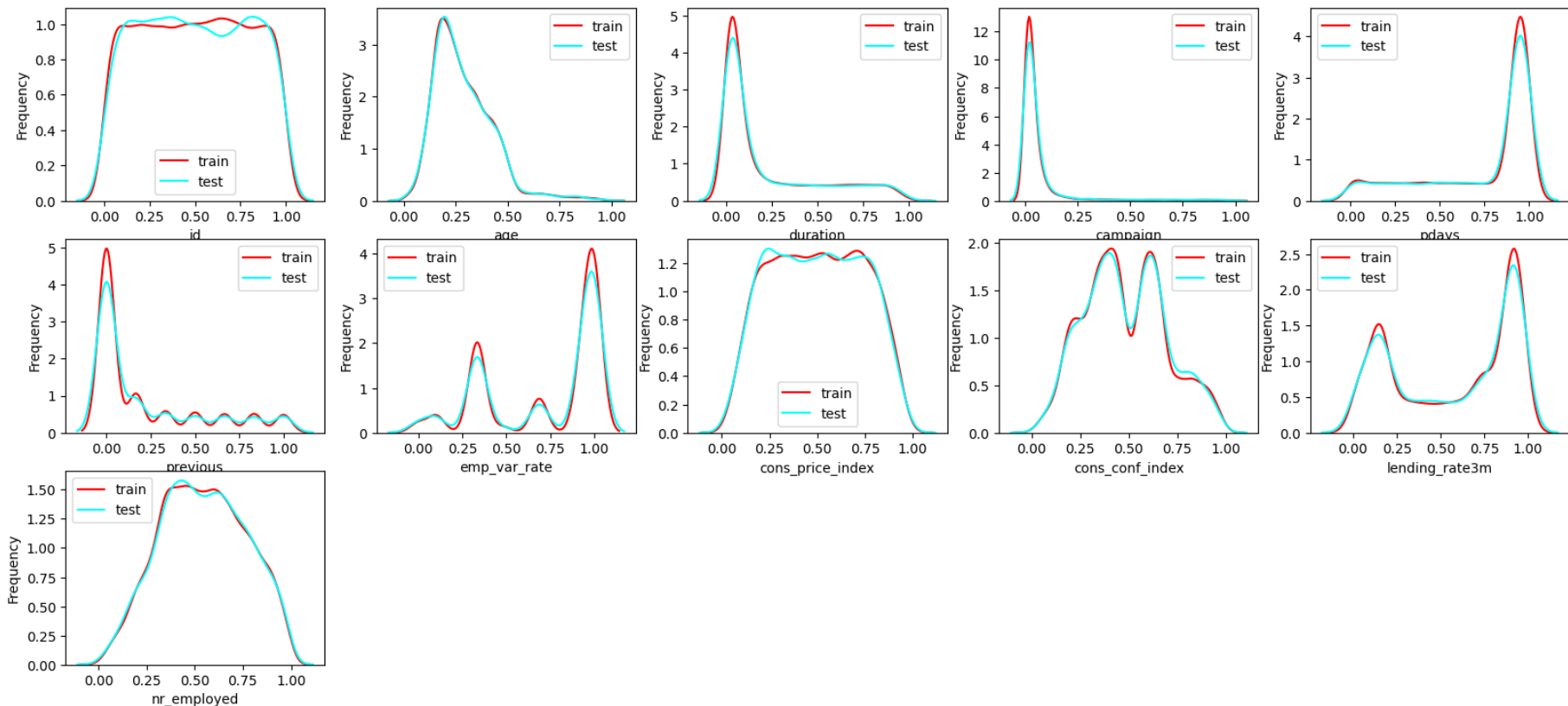
05

复杂特征关系问题

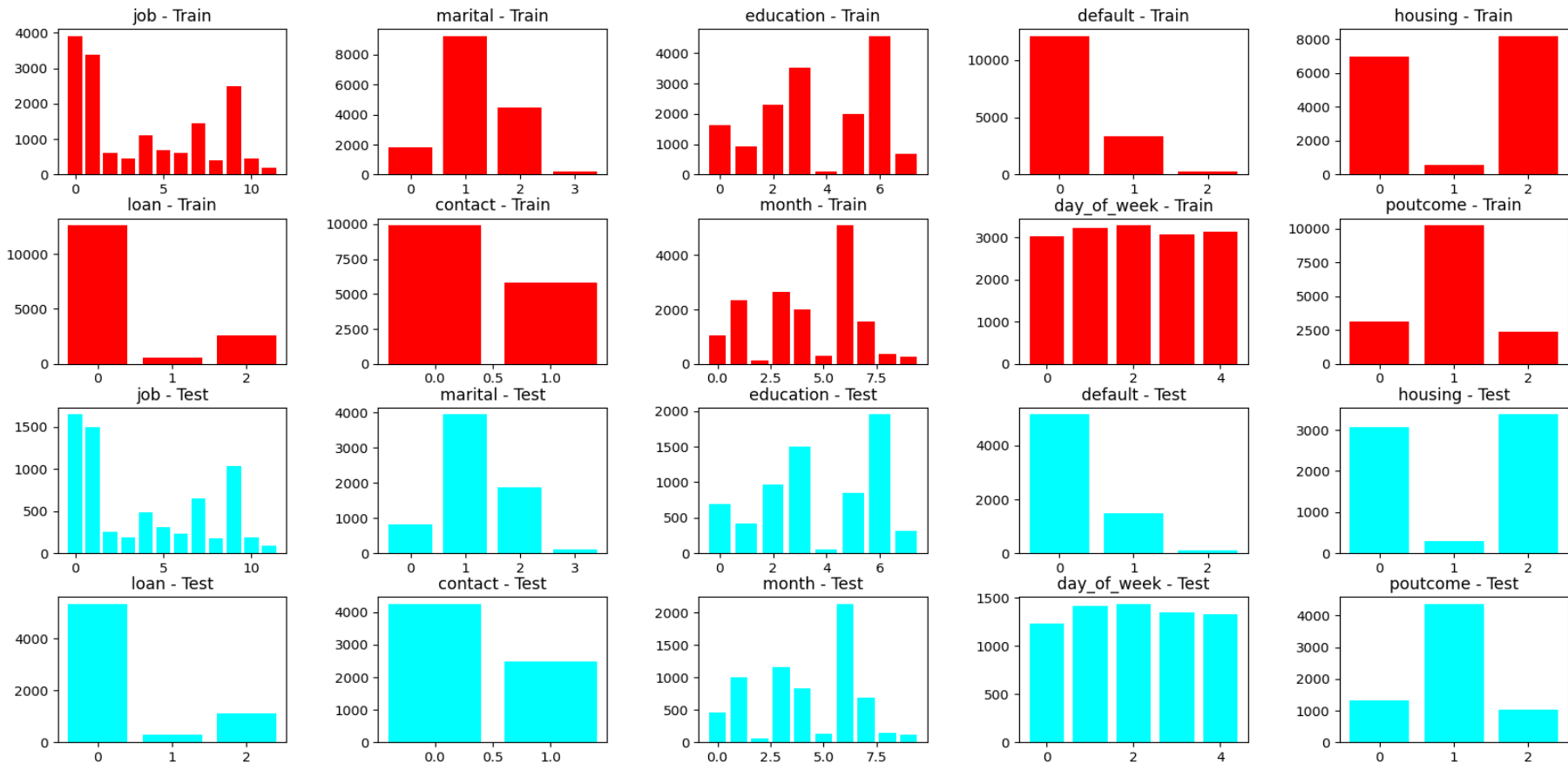


查看特征关系

绘制训练集和测试集中数值变量的KDE图，观察训练集和测试集两者的分布非常接近，得到他们的数据模式是相似的。模型可能具有较好的泛化能力。

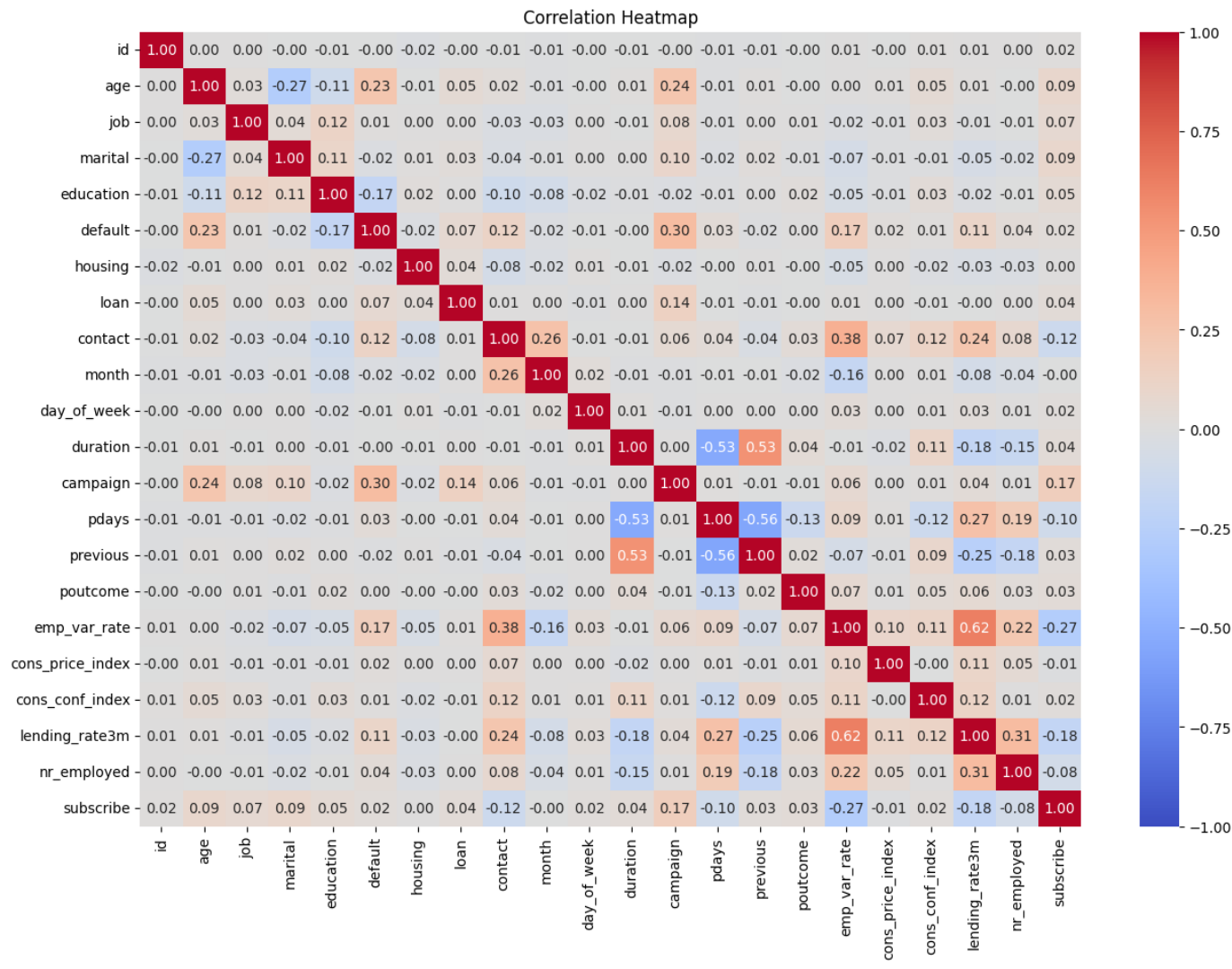


查看特征关系



查看线性关系

根据提供的相关热力图，确实存在一些特征之间的高度相关性，这可能导致多重线性问题。而这会影响Logistic回归的准确率，我们可以使用LightGBM模型进行构建提高准确率。



LightGBM训练

01

模型构建与参数设置

```
2 gbm = LGBMClassifier(n_estimators=600, learning_rate=0.01,  
3                       boosting_type='gbdt',  
                       objective='binary', max_depth=-1,  
                       random_state=2022, metric='auc', force_col_wise='true')
```

02

交叉验证设置参数

```
result1 = []  
mean_score1 = 0  
n_folds = 5  
kf = KFold(n_splits=n_folds, shuffle=True, random_state=2022)
```

03

开始训练

```
for fold, (train_index, test_index) in enumerate(kf.split(X_resampled), 1):  
    x_train_fold = X_resampled.iloc[train_index]  
    y_train_fold = Y_resampled.iloc[train_index]  
    x_test_fold = X_resampled.iloc[test_index]  
    y_test_fold = Y_resampled.iloc[test_index]  
  
    # 在当前折上训练模型  
    gbm.fit(x_train_fold, y_train_fold)
```

LightGBM训练分析

平均验证集AUC: 0.8894279288806481

```
[LightGBM] [Info] Number of positive: 1653, number of negative: 10947
[LightGBM] [Info] Total Bins 1729
[LightGBM] [Info] Number of data points in the train set: 12600, number of used features: 20
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.131190 -> initscore=-1.890474
[LightGBM] [Info] Start training from score -1.890474
```

第 1 折 训练集准确率: 0.928968253968254

第 1 折 训练集AUC: 0.956372813386569

第 1 折 验证集AUC: 0.880417708455055

第 1 折 Confusion Matrix:

```
[[2639  102]
```

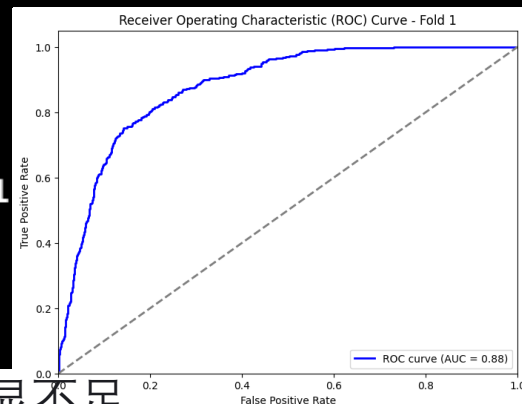
```
 [ 271  138]]
```

第 1 折 类别 0 精确率: 0.91, 召回率: 0.96, F1 分数: 0.93, 支持度: 2741

第 1 折 类别 1 精确率: 0.57, 召回率: 0.34, F1 分数: 0.43, 支持度: 409

第 1 折 准确率: 0.88

第 1 折 对数损失: 0.2648



- 模型在处理负类样本时表现良好，但在识别正类样本时存在明显不足。
- 高训练集AUC和低验证集AUC表明可能存在过拟合问题。
- 需要进一步调整模型参数或采用其他技术，以提高模型在未见过数据上的泛化能力。

06

数据分布失衡问题



过采样技术

01

随机过采样

随机过采样是一种简单的过采样技术，通过随机复制少数类样本来增加其数量，以平衡数据集。这种方法易于实现，但可能因重复数据而导致模型过拟合。

02

SMOTE技术

SMOTE是一种先进的过采样方法，通过在少数类样本间生成新的合成样本来增加少数类的数量，有效减少过拟合并提升模型泛化能力。

03

过采样效果评估

应用随机过采样技术后，AUC值从0.889提升至0.93，表明模型在区分正负样本的能力上有所增强。

07

模型评估



AUC值

Logistic回归模型的AUC值

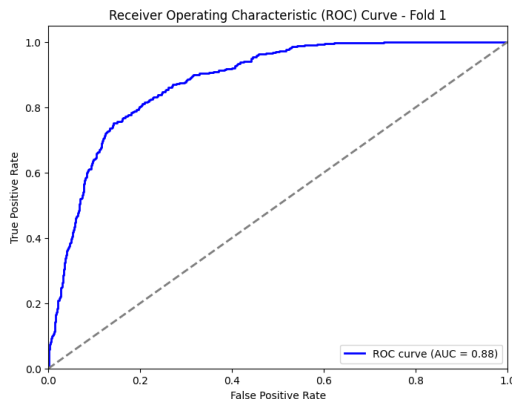
使用Logistic回归模型进行训练后，得到的AUC值为0.77，这表明模型有一定区分正负类的能力，但正类召回率为0.04，说明模型大部分识别为负类，而负类在数据集中占比高，所以AUC较高

	precision	recall	f1-score	support
0	0.94	0.71	0.81	3927
1	0.26	0.70	0.38	573
accuracy			0.70	4500
macro avg	0.60	0.70	0.59	4500
weighted avg	0.85	0.70	0.75	4500

0.7536236134942633

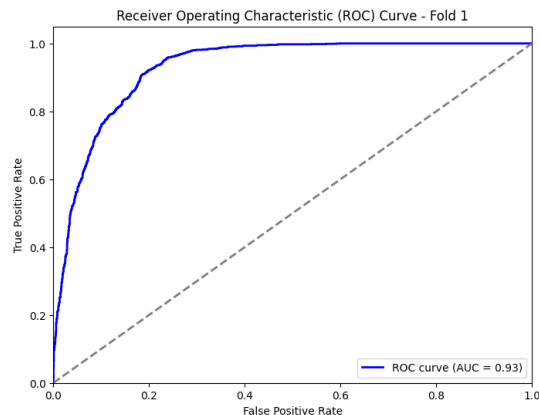
LightGBM模型的AUC值

在使用LightGBM模型进行训练时，各折的AUC值均在0.89左右，这些AUC值表明LightGBM模型在处理复杂非线性关系和特征之间的复杂交互作用方面表现良好。



LightGBM模型的过采样处理后AUC值

对于正类，召回率从0.34提高到0.91，F1分数从0.43提高到0.87，显著提高了模型对正类的识别能力。AUC值从0.89提升到了0.93，表明模型在区分正负样本的能力上有所增强。



第 5 折 Confusion Matrix:

[[2333 445]

[270 2427]]

第 5 折 类别 0 精确率: 0.90, 召回率: 0.84, F1 分数: 0.87, 支持度: 2778

第 5 折 类别 1 精确率: 0.85, 召回率: 0.90, F1 分数: 0.87, 支持度: 2697

第 5 折 准确率: 0.87

第 5 折 对数损失: 0.3253

- 真阴性 (TN) 为2333, 表示有2333个实际为负类的样本被正确预测为负类。
- 假阳性 (FP) 为445, 表示有445个实际为负类的样本被错误预测为正类。
- 真阳性 (TP) 为2427, 表示有2427个实际为正类的样本被正确预测为正类。
- 假阴性 (FN) 为270, 表示有270个实际为正类的样本被错误预测为负类。
- 在所有被预测为负类的样本中, 有90%是真正的负类。在所有实际为负类的样本中, 有84%被正确识别。在所有被预测为正类的样本中, 有85%是真正的正类。在所有实际为正类的样本中, 有90%被正确识别。准确率为0.87, 表示在所有样本中, 有87%被正确分类。

感谢观看

THANKS

